

[text/x-python] collision.py
[text/x-python] game_engine.py
see the pdf for the context and and do this task accordingly: Task 1:
fix the collision bug in `collision.py` and the list-removal loop in
`game\_engine.py`.



Task 1 fix

Both bugs are in the two files you uploaded.

Bug 1, collision.py : `is_caught` only checks the horizontal position, so an object scores the moment it spawns above the basket. It also needs a vertical check.

Bug 2, game_engine.py : the loop calls `self.objects.remove(obj)` while iterating over `self.objects`. Python then skips the element after each removed one, so some objects are missed or double-handled. The fix is to collect the caught objects first and rebuild the list afterwards.

game/collision.py

python

```
"""
collision: figures out whether a falling object is within the basket.
"""

def is_caught(basket_rect, obj):
    """An object is caught only when its position is inside the basket
    rectangle both horizontally AND vertically."""
    within_x = basket_rect.left <= obj.x <= basket_rect.right
    within_y = basket_rect.top <= obj.y <= basket_rect.bottom
    return within_x and within_y
```

game/game_engine.py

Replace the catch-checking loop inside `update()` :

python

```

basket_rect = self.basket.get_rect()

# Split objects into caught and remaining, instead of removing
# from the list while iterating over it.
remaining = []
for obj in self.objects:
    if is_caught(basket_rect, obj):
        self.score += 1
    else:
        remaining.append(obj)
self.objects = remaining

missed = [o for o in self.objects if o.is_past_bottom(HEIGHT)]
if missed:
    self.objects = [o for o in self.objects if not o.is_past_bottom(HEIGHT)]
    self.misses += len(missed)
    if self.misses >= MAX_MISSES:
        self.game_over = True

```

The rest of the file stays the same. The `missed` block is unchanged, and it still works because caught objects are already removed before it runs.

Test it

Run `python3 main.py` and check:

1. Parking the basket under a spawn column no longer scores instantly. The score goes up only when the object visibly touches the basket.
2. Objects that fall past the basket still count as misses.
3. After 5 misses you get Game Over, and R restarts.

Then commit:

```

bash

cd ~/vibecoding/Lab-4/catch-falling
git add .
git commit -m "Task 1: fix collision"

```

I haven't seen `basket.py` or `falling_object.py`. This assumes `get_rect()` returns a `pygame.Rect` (it must, since the original code uses `.left` and `.right`) and that `obj.y` is the object's center. If objects still seem to score slightly early or late, paste those two files and I'll adjust the vertical check to account for the object's size.

When Task 1 works, send me `basket.py` and we'll do Task 2.

give the entire code modified of game_engine.py



Here is the full `game/game_engine.py` with the Task 1 fix applied. The only change is the catch-checking loop in `update()` .

python

```
"""
GameEngine: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic
level (Tasks 2 and 3 ask you to improve them), there's no speed boost
yet (Task 4 builds it from scratch), and catch detection has two
known bugs (see game/collision.py and the catch-checking loop below)
that Task 1 asks you to fix.
"""

import random
import pygame

from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 50
MAX_MISSES = 5

class GameEngine:
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
        self.frames_until_spawn = 0
        self.score = 0
        self.misses = 0
        self.game_over = False

    def _spawn_object(self):
        x = random.randint(20, WIDTH - 20)
        self.objects.append(FallingObject(x=x, y=-14, speed=3))
```

```

def handle_input(self, keys_pressed):
    if self.game_over:
        return
    if keys_pressed[pygame.K_LEFT]:
        self.basket.x -= self.basket.speed
    if keys_pressed[pygame.K_RIGHT]:
        self.basket.x += self.basket.speed
    # Boundary handling: only clamps against the screen edges, not
    # accounting for the basket's own width - it can hang half off
    # either side of the screen.
    self.basket.x = max(0, min(WIDTH, self.basket.x))

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()

def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        self._spawn_object()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for obj in self.objects:
        obj.update()

    basket_rect = self.basket.get_rect()

    # Task 1 fix: split objects into caught and remaining instead of
    # removing from the list while iterating over it.
    remaining = []
    for obj in self.objects:
        if is_caught(basket_rect, obj):
            self.score += 1
        else:
            remaining.append(obj)
    self.objects = remaining

    missed = [o for o in self.objects if o.is_past_bottom(HEIGHT)]
    if missed:
        self.objects = [o for o in self.objects if not o.is_past_bottom(HEIGHT)]
        self.misses += len(missed)
        if self.misses >= MAX_MISSES:
            self.game_over = True

def draw(self, surface, font):
    from game import renderer

```

```

        renderer.draw_scene(surface, self.basket, self.objects)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
        renderer.draw_text(surface, font, f"Misses: {self.misses}/{MAX_MISSES}", (10, 36))

    if self.game_over:
        renderer.draw_banner(surface, font, f"Game Over! Final score: {self.score}. Press R
to restart.")

```

I left the header docstring and the boundary comment in `handle_input` as they were, since Tasks 2 and 3 still need them. You can update the docstring later if you want it to reflect the fixed state.

Remember that this fix works together with the updated `collision.py` from my previous message. Both files need to be changed for the "score only when it reaches the basket" behavior to work.

2026-09-30 11:52:41 You 

[text/x-python] basket.py

1. Task 2: fix the basket boundaries and smooth movement in ``basket.py``.
2. Task 3: varied spawning with an on-screen cap.
3. Task 4: speed boost with a visible indicator. complete these tasks also



Here are Tasks 2, 3 and 4. Only two files change: `basket.py` and `game_engine.py`. `collision.py` stays as it was after Task 1.

I haven't seen `main.py`, `renderer.py` or `falling_object.py`, so I assumed three things:

- `main.py` calls `handle_input(keys)` every frame and `handle_keydown(key)` on key presses. Restart on R already relies on the second, and the starter's basket movement relies on the first.
- `FallingObject(x, y, speed)` accepts any float speed.
- `renderer.draw_text(surface, font, text, (x, y))` works as it does in the starter.

game/basket.py (Tasks 2 and 4)

python

```

"""
Basket: the player-controlled catcher at the bottom of the screen.

```

Task 2: movement is velocity-based (accelerates and slides to a stop),
and the basket is clamped so it never goes past either edge.

Task 4: a timed speed boost with a cooldown.

"""

```
import pygame
```

```
ACCELERATION = 1.0
```

```
FRICTION = 0.80
```

```
BOOST_DURATION_FRAMES = 120      # 2 seconds at 60 FPS
```

```
BOOST_COOLDOWN_FRAMES = 240     # 4 seconds at 60 FPS
```

```
BOOST_MULTIPLIER = 1.8
```

```
class Basket:
```

```
    def __init__(self, x, y, width=90, height=24, speed=5):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.speed = speed
```

```
        self.velocity = 0.0
```

```
        self.boosted_frames = 0
```

```
        self.boost_cooldown = 0
```

```
        self.normal_speed = speed
```

```
        self.boost_speed = speed * BOOST_MULTIPLIER
```

```
    # ---- Task 4: boost ----
```

```
    @property
```

```
    def is_boosted(self):
```

```
        return self.boosted_frames > 0
```

```
    def activate_boost(self):
```

```
        """Start a boost if one is not already running or recharging."""
```

```
        if self.boosted_frames == 0 and self.boost_cooldown == 0:
```

```
            self.boosted_frames = BOOST_DURATION_FRAMES
```

```
    def _tick_boost(self):
```

```
        if self.boosted_frames > 0:
```

```
            self.boosted_frames -= 1
```

```
            if self.boosted_frames == 0:
```

```
                self.boost_cooldown = BOOST_COOLDOWN_FRAMES
```

```
        elif self.boost_cooldown > 0:
```

```
            self.boost_cooldown -= 1
```

```
        self.speed = self.boost_speed if self.is_boosted else self.normal_speed
```

```
    # ---- Task 2: movement ----
```

```
    def update(self, direction, screen_width):
```

```

"""direction: -1 (left), 0 (none) or +1 (right)."""
self._tick_boost()

if direction != 0:
    self.velocity += direction * ACCELERATION
    self.velocity = max(-self.speed, min(self.speed, self.velocity))
else:
    self.velocity *= FRICTION
    if abs(self.velocity) < 0.1:
        self.velocity = 0.0

self.x += self.velocity

# Clamp using the basket's own half-width so it stays fully on screen.
half = self.width / 2
if self.x < half:
    self.x = half
    self.velocity = 0.0
elif self.x > screen_width - half:
    self.x = screen_width - half
    self.velocity = 0.0

def get_rect(self):
    return pygame.Rect(
        int(self.x - self.width / 2), int(self.y - self.height / 2),
        self.width, self.height,
    )

```

game/game_engine.py (Tasks 1, 2, 3 and 4)

python

```

"""
GameEngine: owns the basket and all falling objects.

Task 1: catch detection fixed (collision.py + safe list handling).
Task 2: basket movement/boundaries handled in Basket.update().
Task 3: varied spawn position, speed and timing, with an on-screen cap.
Task 4: speed boost (Space) with an on-screen indicator.
"""

import random
import pygame

from game.basket import (
    Basket,
    BOOST_DURATION_FRAMES,
    BOOST_COOLDOWN_FRAMES,

```

```

)
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT

MAX_MISSES = 5

# Task 3: spawning
MIN_SPAWN_INTERVAL = 25
MAX_SPAWN_INTERVAL = 70
MIN_OBJECT_SPEED = 2.0
MAX_OBJECT_SPEED = 5.0
MAX_OBJECTS_ON_SCREEN = 6
MIN_SPAWN_X_GAP = 60          # avoid spawning right on top of the previous one
SPAWN_RETRY_FRAMES = 10      # when the cap is hit, check again soon

class GameEngine:
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
        self.frames_until_spawn = 0
        self.last_spawn_x = None
        self.score = 0
        self.misses = 0
        self.game_over = False

    # ---- Task 3: spawning ----
    def _pick_spawn_x(self):
        x = random.randint(20, WIDTH - 20)
        for _ in range(10):
            if self.last_spawn_x is None or abs(x - self.last_spawn_x) >= MIN_SPAWN_X_GAP:
                break
            x = random.randint(20, WIDTH - 20)
        return x

    def _spawn_object(self):
        x = self._pick_spawn_x()
        speed = random.uniform(MIN_OBJECT_SPEED, MAX_OBJECT_SPEED)
        self.objects.append(FallingObject(x=x, y=-14, speed=speed))
        self.last_spawn_x = x

    # ---- input ----
    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        direction = 0
        if keys_pressed[pygame.K_LEFT]:
            direction -= 1

```



```

if keys_pressed[pygame.K_RIGHT]:
    direction += 1
self.basket.update(direction, WIDTH)

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()
    elif not self.game_over and key == pygame.K_SPACE:
        self.basket.activate_boost()

# ---- game loop ----
def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        if len(self.objects) < MAX_OBJECTS_ON_SCREEN:
            self._spawn_object()
            self.frames_until_spawn = random.randint(
                MIN_SPAWN_INTERVAL, MAX_SPAWN_INTERVAL
            )
        else:
            self.frames_until_spawn = SPAWN_RETRY_FRAMES

    for obj in self.objects:
        obj.update()

    basket_rect = self.basket.get_rect()

    # Task 1: split into caught/remaining instead of removing while iterating.
    remaining = []
    for obj in self.objects:
        if is_caught(basket_rect, obj):
            self.score += 1
        else:
            remaining.append(obj)
    self.objects = remaining

    missed = [o for o in self.objects if o.is_past_bottom(HEIGHT)]
    if missed:
        self.objects = [o for o in self.objects if not o.is_past_bottom(HEIGHT)]
        self.misses += len(missed)
        if self.misses >= MAX_MISSES:
            self.game_over = True

# ---- drawing ----
def _draw_boost_indicator(self, surface, font):
    from game import renderer

```

```

b = self.basket
bar_x, bar_y, bar_w, bar_h = 10, 66, 140, 12

if b.is_boosted:
    label = "BOOST ACTIVE!"
    fraction = b.boosted_frames / BOOST_DURATION_FRAMES
    color = (255, 200, 0)
    # Gold outline around the basket while boosted.
    pygame.draw.rect(surface, color, b.get_rect().inflate(6, 6), 3)
elif b.boost_cooldown > 0:
    label = "Boost recharging..."
    fraction = 1 - b.boost_cooldown / BOOST_COOLDOWN_FRAMES
    color = (150, 150, 150)
else:
    label = "Boost READY (Space)"
    fraction = 1.0
    color = (80, 220, 120)

renderer.draw_text(surface, font, label, (10, 46))
pygame.draw.rect(surface, (60, 60, 60), (bar_x, bar_y, bar_w, bar_h))
pygame.draw.rect(surface, color, (bar_x, bar_y, int(bar_w * fraction), bar_h))
pygame.draw.rect(surface, (255, 255, 255), (bar_x, bar_y, bar_w, bar_h), 1)

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.basket, self.objects)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Misses: {self.misses}/{MAX_MISSES}", (10, 26))
    self._draw_boost_indicator(surface, font)

    if self.game_over:
        renderer.draw_banner(surface, font, f"Game Over! Final score: {self.score}. Press R to restart.")

```

One layout note: in the starter, the "Misses" text sat at y=36. I moved it to y=26 and put the boost label at y=46 and the bar at y=66. If the text overlaps on your screen (it depends on the font size), increase the spacing.

How to test and commit (one task at a time)

Task 2: Hold Left and Right. The basket should speed up gradually, slide slightly when you release, and stop with its whole body on screen at both edges.

```
bash
```

```

git add .
git commit -m "Task 2: fix basket boundaries and smooth movement"

```

Task 3: Objects should appear at different columns, fall at different speeds, and arrive at irregular intervals. Never more than 6 should be on screen at once. To check the cap, temporarily set `MAX_OBJECTS_ON_SCREEN = 1` and confirm it holds.

```
bash
```

```
git add .  
git commit -m "Task 3: varied spawning with on-screen cap"
```

Task 4: Press Space. The label should change to "BOOST ACTIVE!", the bar should drain, the basket should get a gold outline and move faster, and then it should expire. After that comes "Boost recharging..." and finally "Boost READY (Space)".

```
bash
```

```
git add .  
git commit -m "Task 4: speed boost with visible indicator"
```

Since you're working through the tasks one at a time, you can also apply the changes in stages. Task 2 needs only the movement parts of the two files, but pasting both files in full and committing once at the end works too.

If something breaks, paste the exact error or describe what you see. A likely cause is `main.py` not calling `handle_input` every frame, in which case send me `main.py` and I'll adjust. When everything works, record the "after" video as described in the earlier steps.

2026-09-30 11:54:24 You 

[text/x-python] falling_object.py
[text/x-python] renderer.py
[text/x-python] main.py



I checked the three files against what I assumed earlier. `main.py` calls `handle_input` every frame and `handle_keydown` on key presses, `draw_text` has the signature I used, and `FallingObject` accepts float speeds. The logic works as written, and even at the maximum speed of 5 an object can't skip over the 24 px tall basket in one frame.

I did find one mistake in my earlier code, in the HUD spacing. Your font is size 22, so each line is about 25 px tall. I put Score at y=10 and Misses at y=26, so those two lines would overlap, and the boost label would sit too close to them. The starter's 26 px spacing was correct, so I should have kept it.

Fix in `game/game_engine.py`

Replace `_draw_boost_indicator` and `draw` with these:

python

```
def _draw_boost_indicator(self, surface, font):
    from game import renderer
    b = self.basket
    bar_x, bar_y, bar_w, bar_h = 10, 90, 140, 12

    if b.is_boosted:
        label = "BOOST ACTIVE!"
        fraction = b.boosted_frames / BOOST_DURATION_FRAMES
        color = (255, 200, 0)
        # Gold outline around the basket while boosted.
        pygame.draw.rect(surface, color, b.get_rect().inflate(6, 6), 3)
    elif b.boost_cooldown > 0:
        label = "Boost recharging..."
        fraction = 1 - b.boost_cooldown / BOOST_COOLDOWN_FRAMES
        color = (150, 150, 150)
    else:
        label = "Boost READY (Space)"
        fraction = 1.0
        color = (80, 220, 120)

    renderer.draw_text(surface, font, label, (10, 62), color)
    pygame.draw.rect(surface, (60, 60, 60), (bar_x, bar_y, bar_w, bar_h))
    pygame.draw.rect(surface, color, (bar_x, bar_y, int(bar_w * fraction), bar_h))
    pygame.draw.rect(surface, (255, 255, 255), (bar_x, bar_y, bar_w, bar_h), 1)

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.basket, self.objects)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Misses: {self.misses}/{MAX_MISSES}", (10, 36))
    self._draw_boost_indicator(surface, font)

    if self.game_over:
        renderer.draw_banner(surface, font, f"Game Over! Final score: {self.score}. Press R to restart.")
```

The layout is now Score at y=10, Misses at y=36, the boost label at y=62 and the bar at y=90. The label also takes the state's color, so green means ready, gold means active and grey means recharging. That reads better in the video.

Optional: `main.py`

Update the docstring so the controls are documented:



```
python
```

Controls: Left/Right arrows to move the basket, Space **for** a speed boost.

Nothing else in `main.py` , `renderer.py` , `falling_object.py` , `basket.py` or `collision.py` needs to change.

Before you record the "after" video

1. Run `python3 main.py` and confirm the text lines don't overlap.
2. Do a quick pass through all four behaviors. The score should rise only when a ball touches the basket, the basket should stop fully on screen at both edges, spawns should vary, and Space should trigger the boost and its indicator.
3. Commit:

```
bash
```

```
cd ~/vibecoding
git add .
git commit -m "Fix HUD spacing for boost indicator"
```

One thing to know: a ball only counts as caught when its center is over the basket, so a ball that only clips the edge falls past. That's fine for this lab, but if it feels harsh in play, say so and I'll switch it to a circle-overlap check.

2026-09-30 11:55:21 You 

give all the code full version of all the modofications proerly.



Here are the complete final versions of every file that changed. Copy each one over your existing file. `falling_object.py` and `renderer.py` are unchanged.

game/collision.py

```
python
```

```
"""
collision: figures out whether a falling object is within the basket.
"""
```

```
def is_caught(basket_rect, obj):
    """An object is caught only when its position is inside the basket
    rectangle both horizontally AND vertically."""
    within_x = basket_rect.left <= obj.x <= basket_rect.right
    within_y = basket_rect.top <= obj.y <= basket_rect.bottom
    return within_x and within_y
```

game/basket.py

python

```
"""
Basket: the player-controlled catcher at the bottom of the screen.

Task 2: movement is velocity-based (accelerates and slides to a stop),
        and the basket is clamped so it never goes past either edge.
Task 4: a timed speed boost with a cooldown.
"""

import pygame

ACCELERATION = 1.0
FRICTION = 0.80

BOOST_DURATION_FRAMES = 120      # 2 seconds at 60 FPS
BOOST_COOLDOWN_FRAMES = 240     # 4 seconds at 60 FPS
BOOST_MULTIPLIER = 1.8

class Basket:
    def __init__(self, x, y, width=90, height=24, speed=5):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.speed = speed
        self.velocity = 0.0
        self.boosted_frames = 0
        self.boost_cooldown = 0
        self.normal_speed = speed
        self.boost_speed = speed * BOOST_MULTIPLIER

    # ---- Task 4: boost ----
    @property
    def is_boosted(self):
        return self.boosted_frames > 0
```

```

def activate_boost(self):
    """Start a boost if one is not already running or recharging."""
    if self.boosted_frames == 0 and self.boost_cooldown == 0:
        self.boosted_frames = BOOST_DURATION_FRAMES

def _tick_boost(self):
    if self.boosted_frames > 0:
        self.boosted_frames -= 1
        if self.boosted_frames == 0:
            self.boost_cooldown = BOOST_COOLDOWN_FRAMES
    elif self.boost_cooldown > 0:
        self.boost_cooldown -= 1
    self.speed = self.boost_speed if self.is_boosted else self.normal_speed

# ---- Task 2: movement ----
def update(self, direction, screen_width):
    """direction: -1 (left), 0 (none) or +1 (right)."""
    self._tick_boost()

    if direction != 0:
        self.velocity += direction * ACCELERATION
        self.velocity = max(-self.speed, min(self.speed, self.velocity))
    else:
        self.velocity *= FRICTION
        if abs(self.velocity) < 0.1:
            self.velocity = 0.0

    self.x += self.velocity

    # Clamp using the basket's own half-width so it stays fully on screen.
    half = self.width / 2
    if self.x < half:
        self.x = half
        self.velocity = 0.0
    elif self.x > screen_width - half:
        self.x = screen_width - half
        self.velocity = 0.0

def get_rect(self):
    return pygame.Rect(
        int(self.x - self.width / 2), int(self.y - self.height / 2),
        self.width, self.height,
    )

```

game/game_engine.py

python

```
"""
```

GameEngine: owns the basket and all falling objects.

Task 1: catch detection fixed (collision.py + safe list handling).

Task 2: basket movement/boundaries handled in Basket.update().

Task 3: varied spawn position, speed and timing, with an on-screen cap.

Task 4: speed boost (Space) with an on-screen indicator.

```
"""
```

```
import random
```

```
import pygame
```

```
from game.basket import (
    Basket,
    BOOST_DURATION_FRAMES,
    BOOST_COOLDOWN_FRAMES,
)
```

```
from game.falling_object import FallingObject
```

```
from game.collision import is_caught
```

```
from game.renderer import WIDTH, HEIGHT
```

```
MAX_MISSES = 5
```

```
# Task 3: spawning
```

```
MIN_SPAWN_INTERVAL = 25
```

```
MAX_SPAWN_INTERVAL = 70
```

```
MIN_OBJECT_SPEED = 2.0
```

```
MAX_OBJECT_SPEED = 5.0
```

```
MAX_OBJECTS_ON_SCREEN = 6
```

```
MIN_SPAWN_X_GAP = 60 # avoid spawning right on top of the previous one
```

```
SPAWN_RETRY_FRAMES = 10 # when the cap is hit, check again soon
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
```

```
        self.objects = []
```

```
        self.frames_until_spawn = 0
```

```
        self.last_spawn_x = None
```

```
        self.score = 0
```

```
        self.misses = 0
```

```
        self.game_over = False
```

```
# ---- Task 3: spawning ----
```

```
    def _pick_spawn_x(self):
```

```
        x = random.randint(20, WIDTH - 20)
```

```
        for _ in range(10):
```

```
            if self.last_spawn_x is None or abs(x - self.last_spawn_x) >= MIN_SPAWN_X_GAP:
```



```

        break
    x = random.randint(20, WIDTH - 20)
    return x

def _spawn_object(self):
    x = self._pick_spawn_x()
    speed = random.uniform(MIN_OBJECT_SPEED, MAX_OBJECT_SPEED)
    self.objects.append(FallingObject(x=x, y=-14, speed=speed))
    self.last_spawn_x = x

# ---- input ----
def handle_input(self, keys_pressed):
    if self.game_over:
        return
    direction = 0
    if keys_pressed[pygame.K_LEFT]:
        direction -= 1
    if keys_pressed[pygame.K_RIGHT]:
        direction += 1
    self.basket.update(direction, WIDTH)

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()
    elif not self.game_over and key == pygame.K_SPACE:
        self.basket.activate_boost()

# ---- game loop ----
def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        if len(self.objects) < MAX_OBJECTS_ON_SCREEN:
            self._spawn_object()
            self.frames_until_spawn = random.randint(
                MIN_SPAWN_INTERVAL, MAX_SPAWN_INTERVAL
            )
        else:
            self.frames_until_spawn = SPAWN_RETRY_FRAMES

    for obj in self.objects:
        obj.update()

    basket_rect = self.basket.get_rect()

    # Task 1: split into caught/remaining instead of removing while iterating.
    remaining = []

```

```

for obj in self.objects:
    if is_caught(basket_rect, obj):
        self.score += 1
    else:
        remaining.append(obj)
self.objects = remaining

missed = [o for o in self.objects if o.is_past_bottom(HEIGHT)]
if missed:
    self.objects = [o for o in self.objects if not o.is_past_bottom(HEIGHT)]
    self.misses += len(missed)
    if self.misses >= MAX_MISSES:
        self.game_over = True

# ---- drawing ----
def _draw_boost_indicator(self, surface, font):
    from game import renderer
    b = self.basket
    bar_x, bar_y, bar_w, bar_h = 10, 90, 140, 12

    if b.is_boosted:
        label = "BOOST ACTIVE!"
        fraction = b.boosted_frames / BOOST_DURATION_FRAMES
        color = (255, 200, 0)
        # Gold outline around the basket while boosted.
        pygame.draw.rect(surface, color, b.get_rect().inflate(6, 6), 3)
    elif b.boost_cooldown > 0:
        label = "Boost recharging..."
        fraction = 1 - b.boost_cooldown / BOOST_COOLDOWN_FRAMES
        color = (150, 150, 150)
    else:
        label = "Boost READY (Space)"
        fraction = 1.0
        color = (80, 220, 120)

    renderer.draw_text(surface, font, label, (10, 62), color)
    pygame.draw.rect(surface, (60, 60, 60), (bar_x, bar_y, bar_w, bar_h))
    pygame.draw.rect(surface, color, (bar_x, bar_y, int(bar_w * fraction), bar_h))
    pygame.draw.rect(surface, (255, 255, 255), (bar_x, bar_y, bar_w, bar_h), 1)

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.basket, self.objects)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Misses: {self.misses}/{MAX_MISSES}", (10, 36))
    self._draw_boost_indicator(surface, font)

    if self.game_over:
        renderer.draw_banner(surface, font, f"Game Over! Final score: {self.score}. Press R

```

```
to restart.")
```

main.py

Only the docstring changed:

python

```
"""
Catch the Falling Objects (Lab Starter)

Run with: python3 main.py

Controls: Left/Right arrows to move the basket, Space for a speed boost.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Catch the Falling Objects")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                engine.handle_keydown(event.key)

        keys = pygame.key.get_pressed()
        engine.handle_input(keys)
        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()
```

```
if __name__ == "__main__":  
    main()
```

Test and commit

bash

```
cd ~/vibecoding/Lab-4/catch-falling  
python3 main.py
```

Check that:

1. Score goes up only when a ball reaches the basket.
2. The basket stops fully on screen at both edges while you hold the key.
3. Balls vary in position, speed and timing, with at most 6 on screen.
4. Space shows the gold "BOOST ACTIVE!" bar, then grey "Boost recharging...", then green "Boost READY (Space)".

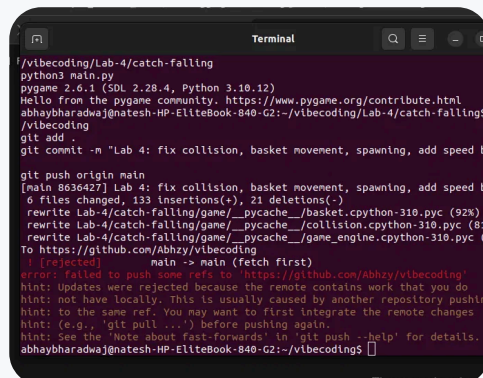
Then commit and push:

bash

```
cd ~/vibecoding  
git add .  
git commit -m "Lab 4: fix collision, basket movement, spawning, add speed boost"  
git push origin main
```

Once it all works, record the `after.webm` video.

2026-09-30 11:59:16 You 



```
Terminal  
/vibecoding/Lab-4/catch-falling  
python3 main.py  
pygame 2.6.1 (SDL 2.28.4, Python 3.10.12)  
Hello from the pygame community. https://www.pygame.org/contribute.html  
abhaybharadwaj@natesh-HP-EliteBook-840-G2:~/vibecoding/Lab-4/catch-falling$  
/vibecoding  
git add .  
git commit -m "Lab 4: fix collision, basket movement, spawning, add speed boost"  
git push origin main  
[main 8636427] Lab 4: fix collision, basket movement, spawning, add speed boost  
6 files changed, 133 insertions(+), 21 deletions(-)  
rewrite Lab-4/catch-falling/game/_pycache_/basket.cpython-310.pyc (92%)  
rewrite Lab-4/catch-falling/game/_pycache_/collision.cpython-310.pyc (81%)  
rewrite Lab-4/catch-falling/game/_pycache_/game_engine.cpython-310.pyc (50%)  
To https://github.com/Abhzy/vibecoding  
 ! [rejected]        main -> main (fetch first)  
error: failed to push some refs to 'https://github.com/Abhzy/vibecoding'  
hint: Updates were rejected because the remote contains work that you do  
hint: not have locally. This is usually caused by another repository pushing  
hint: to the same ref. You may want to first integrate the remote changes  
hint: (e.g., 'git pull ...') before pushing again.  
hint: See the 'Note about fast-forwards' in 'git push --help' for details.  
abhaybharadwaj@natesh-HP-EliteBook-840-G2:~/vibecoding$
```

Your commit worked. Only the push was rejected, because the remote `main` has a commit your local copy doesn't. That's probably the `before.webm` video you uploaded through the GitHub website, since your local repo never received it. Nothing is lost. You just need to pull that commit in first.

Run this:

```
bash

cd ~/vibecoding
git pull --no-rebase origin main
```

Then push:

```
bash

git push origin main
```

If a text editor opens asking for a merge commit message (nano or vim), just save and exit the editor:

- nano: press `Ctrl + O` , then `Enter` , then `Ctrl + X`
- vim: type `:wq` and press `Enter`

If you get a merge conflict, it will say `CONFLICT` in the output. That's unlikely here, since the remote change is probably just the added video file. If it does happen, paste the output here and I'll walk you through it.

Check the result. After the pull, run `ls Lab-4/videos` and you should see `before.webm` there. Then you'll be ready to record `after.webm` .

One more thing I noticed in your output: the commit includes `__pycache__/*.pyc` files. They're harmless, but they clutter the repo. To stop tracking them, run this after your push succeeds:

```
bash

cd ~/vibecoding
echo "__pycache__/" >> .gitignore
git rm -r --cached Lab-4/catch-falling/game/__pycache__
git add .
git commit -m "Ignore pycache"
git push origin main
```

This step is optional, so skip it if you'd rather not touch anything else.